

Progetto di Scalable and Cloud Programming

Sara Casadio

Matricola N. 0001186923

Alma Mater Studiorum – Università di Bologna

Anno Accademico 2025–2026

1 Introduzione

Il progetto ha come obiettivo la realizzazione di un'implementazione in Scala e Spark per l'analisi di co-occorrenza di terremoti in località diverse, condotta su un dataset specifico. L'analisi consiste nel ricercare la coppia di località che presenta il massimo numero di co-occorrenze di eventi sismici nell'arco di un giorno, ed elencare in ordine crescente le date in cui tali eventi co-occorrono. L'elaborazione segue il paradigma map-reduce ed è stata eseguita sul dataset completo tramite cluster Spark gestiti su Google Cloud Dataproc.

1.1 Pre-processing

Ogni evento del CSV viene normalizzato in una coppia (`data`, `cella`):

- le coordinate sono arrotondate alla prima cifra decimale con modalità `HALF_UP` tramite `java.math.BigDecimal`, per evitare le imprecisioni dell'aritmetica in virgola mobile che si manifesterebbero sui valori a metà intervallo;
- dall'istante temporale viene estratta la sola data, così che la finestra di co-occorrenza sia di un giorno;
- le coppie (`data`, `cella`) duplicate vengono eliminate, perché la co-occorrenza conta i giorni in cui due località sono entrambe attive e non i singoli eventi.

Le righe malformate non interrompono l'elaborazione ma vengono scartate dai filtri successivi.

2 Implementazione

2.1 Rappresentazione dei dati

I dati intermedi non usano oggetti ma solo tipi primitivi: ogni cella geografica è racchiusa in un singolo `Int`, mettendo latitudine e longitudine (moltiplicate per dieci e arrotondate) in 16 bit ciascuna, mentre ogni data diventa un `Int` nel formato `YYYYMMDD`.

```
def packCell(latTenths: Int, lonTenths: Int): Int = (latTenths << 16) | (lonTenths & 0xFFFF)
```

2.2 Lettura e aggregazione per data

Il CSV viene letto come `DataFrame` e subito convertito in `RDD`. Una `map` applica la normalizzazione descritta sopra e produce un `RDD` di coppie (`data`, `cella`). Si tratta di una trasformazione narrow, quindi viene eseguita in locale su ogni partizione senza scambio di dati fra i nodi.

Le celle vengono poi raggruppate per data con `combineByKey`, usando un `HashSet[Int]` come accumulatore.

```
.combineByKey(  
  (c: Int) => mutable.HashSet(c), // crea l'accumulatore  
  (set: mutable.HashSet[Int], c: Int) => { set += c; set }, // aggiunge un elemento  
  (a: mutable.HashSet[Int], b: mutable.HashSet[Int]) => { a ++= b; a }, // unisce due accumulatori  
  numPartitions)
```

Le tre funzioni hanno la stessa struttura di **aggregate**: creazione del valore iniziale, inserimento sequenziale all'interno di una partizione e fusione fra partizioni diverse. Rispetto a **groupByKey**, **combineByKey** applica le prime due funzioni localmente prima dello shuffle, quindi la deduplicazione avviene ancora sul nodo di origine. Il numero di dati che attraversa la rete diventa così proporzionale al numero di celle distinte per giorno.

Gli accumulatori sono mutabili, ma ognuno appartiene al task che lo crea e non viene condiviso con gli altri task. Non c'è quindi stato condiviso fra i nodi.

Le date che contengono una sola cella vengono eliminate con un **filter**, perché da una sola cella non si ottiene nessuna coppia. Le celle che restano vengono poi ordinate con **mapValues**. L'ordinamento serve due volte: nella fase successiva permette di generare ogni coppia una sola volta, e in quella finale permette di cercare le celle con una ricerca binaria. Si usa **mapValues** al posto di **map** perché mantiene il partizionamento già deciso da **combineByKey**.

2.3 Conteggio delle co-occorrenze

Per contare le co-occorrenze si è adottato il pattern *stripes*. Per ogni data, e per ogni cella in posizione i dell'array ordinato, viene emessa una sola coppia chiave-valore: la chiave è la cella stessa, il valore è una mappa che associa il conteggio 1 a tutte le celle in posizione $j > i$. Poiché l'array è già ordinato, ogni coppia viene prodotta una volta sola e in forma canonica, senza doverla normalizzare.

```
Iterator.tabulate(n - 1) { i =>
  val stripe = new mutable.HashMap[Int, Int]()
  var j = i + 1
  while (j < n) { stripe += (cells(j) -> 1); j += 1 }
  (cells(i), stripe)
}
```

Le stripe relative alla stessa cella, provenienti da date diverse, vengono fuse da una **reduceByKey** che somma i conteggi chiave per chiave. Anche qui il combiner locale di **reduceByKey** aggrega all'interno della partizione prima dello shuffle.

È stato valutato anche il pattern *pairs*, che per ogni combinazione emette una coppia $((a,b), 1)$, ma è stato scartato perché genera molti più record: per ogni data ne produce uno per ogni coppia di celle, mentre stripes ne produce uno per cella. Con stripes i record scambiati sono meno, anche se più grandi, e parte del lavoro si sposta sulla memoria dei nodi che uniscono le mappe.

2.4 Selezione della coppia e recupero delle date

Il massimo si trova in due passaggi. Prima una **map** cerca il valore più alto dentro ogni stripe, poi una **reduce** confronta questi valori fra loro e restituisce il massimo globale. Il calcolo è distribuito e al driver arriva un solo risultato.

Per ricavare le date della coppia vincente si riusa l'RDD delle celle per data. Ogni giorno viene tenuto solo se contiene entrambe le celle, e la verifica si fa con due ricerche binarie, dato che l'array è già ordinato. In questo modo non serve rifare l'aggregazione, che è la parte più costosa del programma.

2.5 Persistenza

Spark non esegue subito le trasformazioni: aspetta che un'azione ne chieda il risultato. Se però più azioni partono dalla stessa trasformazione, questa viene rifatta da capo ogni volta. Nel programma l'RDD delle celle per data serve due volte, prima per contare le coppie e poi per estrarre le date, quindi viene salvato in memoria con **StorageLevel.MEMORY_AND_DISK**: in questo modo lettura, normalizzazione e raggruppamento vengono eseguiti una sola volta.

2.6 Grado di parallelismo

Il numero di partizioni è passato direttamente alle due trasformazioni wide della pipeline (`combineByKey` e `reduceByKey`) invece di essere imposto con un `repartition` sui dati appena letti. Il parallelismo ottenuto è lo stesso, ma si evita uno shuffle aggiuntivo sull'intero dataset non ancora normalizzato, cioè nel momento in cui i record occupano più spazio perché sono ancora stringhe. Il primo scambio di dati fra nodi avviene così direttamente sulle coppie (`data`, `cella`) già compattate.

3 Risultati dell'analisi

Sul dataset completo la coppia di celle che co-occorre più frequentemente è

$$((38.8, -122.8), (38.8, -122.7))$$

con **10 014 co-occorrenze**, distribuite su date comprese fra il 1990-01-05 e il 2023-07-29. Il risultato è identico in tutte le configurazioni testate.

4 Analisi di scalabilità e prestazioni

4.1 Setup sperimentale

Gli esperimenti sono stati eseguiti sul dataset completo, caricato su un bucket di Google Cloud Storage, con cluster Dataproc creati da riga di comando nella regione `us-east1`. Le macchine usate sono `n2-standard-4` (4 vCPU, 16 GB di RAM), con dischi da 240 GB. Sono state provate tre configurazioni, con 2, 3 e 4 nodi worker, corrispondenti a 8, 12 e 16 core complessivi. Per ogni cluster sono stati testati i valori di partizioni 4, 8, 16, 32, 64, 128 e 256, lasciando la configurazione Spark predefinita di Dataproc. Il tempo riportato è quello misurato dal programma, che parte dopo la creazione della `SparkSession`.

4.2 Tempi misurati

Tabella 1: Tempi di esecuzione (s) su cluster `n2-standard-4`. In grassetto l'ottimo di ciascuna configurazione.

Workers	Partizioni						
	4	8	16	32	64	128	256
2	158,11	112,19	110,27	110,74	112,55	120,85	131,97
3	130,13	110,69	96,30	93,94	99,27	103,55	127,07
4	127,77	110,80	81,23	77,81	79,22	99,35	114,28

4.3 Impatto del numero di partizioni

Le curve mostrano il comportamento a forma di U, ma si può notare che nella colonna a 8 partizioni i tre cluster impiegano circa lo stesso tempo. Con 8 task disponibili e 12 o 16 core, gran parte delle risorse resta inattiva e aggiungere worker non produce vantaggi perché il parallelismo effettivo è limitato dal numero di partizioni, non dall'hardware.

Il caso opposto è quello di avere troppe partizioni che introducono un costo crescente con la dimensione del cluster. Rispetto al proprio tempo migliore, con 256 partizioni si perde il 20% a 2 worker, il 35% a 3 e il 47% a 4. Quando i task sono tanti, infatti, ognuno ha poco lavoro da fare, ma i costi fissi di gestione dei task e di apertura dei blocchi di shuffle restano gli stessi e finiscono per pesare più del lavoro utile. L'ottimo si sposta verso destra al crescere delle risorse: 16 partizioni con 8 core, 32 con 12 e 16 core.

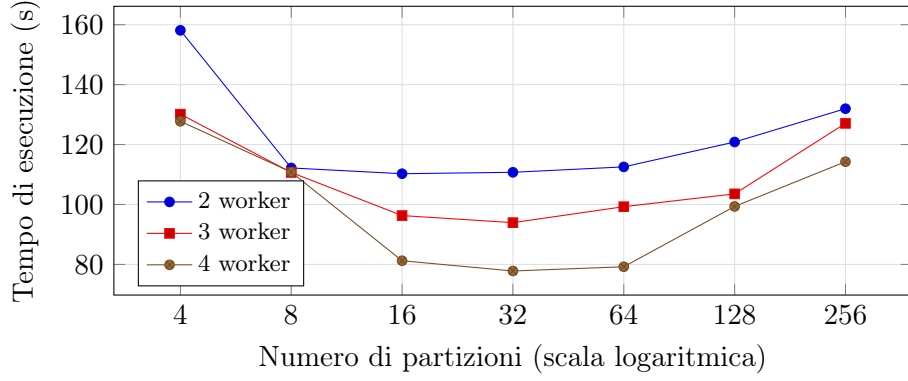


Figura 1: Andamento del tempo al variare del numero di partizioni.

4.4 Strong scaling: speedup ed efficienza

Speedup ed efficienza sono calcolati rispetto al cluster più piccolo testato (2 worker). Indicando con $p = w/2$ il fattore di crescita delle risorse rispetto a questo riferimento, si ha

$$S(w) = \frac{T(2)}{T(w)}, \quad E(w) = \frac{S(w)}{p}.$$

Tabella 2: Speedup ed efficienza, calcolati sul tempo ottimale di ciascun cluster.

Macchina	Workers	Core	Partizioni	Best time (s)	Speedup	Efficienza
n2-standard-4	2	8	16	110,27	1,00	1,000
n2-standard-4	3	12	32	93,94	1,17	0,783
n2-standard-4	4	16	32	77,81	1,42	0,709

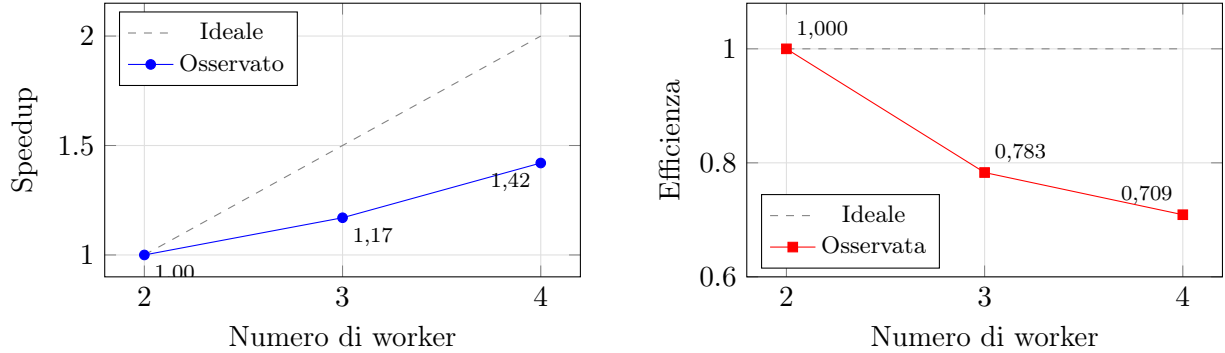


Figura 2: Analisi di strong scaling.

La legge di Amdahl esprime lo speedup in funzione della frazione seriale f come $S = 1/(f + (1 - f)/p)$; invertendo la relazione rispetto a f si ottiene

$$f = \frac{p/S - 1}{p - 1},$$

che applicata ai due punti misurati fornisce $f = 0,56$ con 3 worker ($p = 1,5$) e $f = 0,41$ con 4 worker ($p = 2$), cioè $f \approx 0,49$ in media: una frazione parallelizzabile intorno al 51%.

Questo valore, apparentemente basso, non dipende dall'algoritmo ma da come si compone il tempo di esecuzione. Il lavoro svolto dal programma si divide infatti in due parti: una che si distribuisce fra i worker e diventa più veloce man mano che se ne aggiungono, e una che resta uguale in tutte le configurazioni perché non dipende dal numero di nodi.

Separando le due componenti a partire dai tempi ottimali misurati si ottiene una parte fissa di circa 49 secondi. Per ricavarla basta osservare che, se il tempo si scrive come una parte fissa più una parte che si divide fra i worker, allora riportando i tempi in funzione di $1/w$ i tre punti si dispongono lungo una retta: la retta che meglio li interpola vale 49 secondi nel punto in cui $1/w$ è zero, cioè con un numero infinito di worker, e questo è appunto il tempo che non si riesce a distribuire.

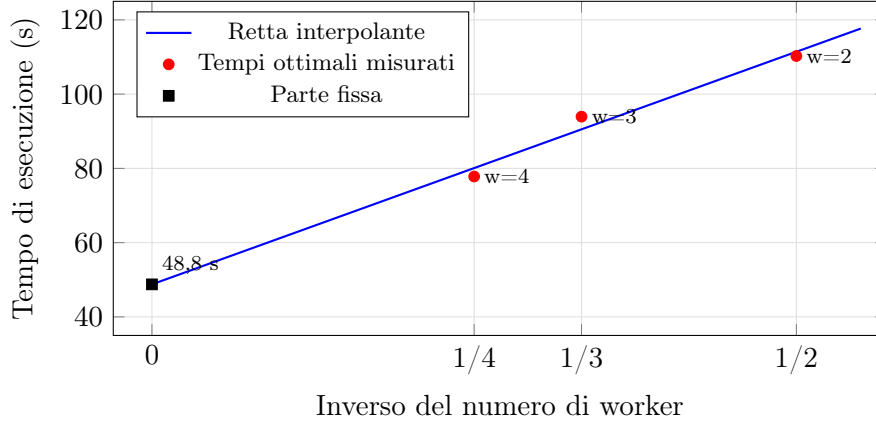


Figura 3: Componente che non si distribuisce fra i worker.

A 2 worker questa componente rappresenta il 44% del tempo totale, un valore molto vicino alla f stimata sopra. Non è quindi l'algoritmo a parallelizzarsi male: il modello che assume una parte perfettamente distribuita riproduce i tempi misurati con scarti di pochi secondi, e sono i costi fissi della piattaforma a pesare sul totale.

5 Conclusioni

L'approccio basato su `combineByKey` con deduplicazione locale e su un conteggio delle co-occorrenze secondo il pattern *stripes* consente di completare l'analisi sull'intero dataset in poco più di un minuto su un cluster di quattro nodi. La scelta di non introdurre un `repartition` iniziale contribuisce a contenere il volume di shuffle e la pressione sulla memoria.

L'analisi di scalabilità mostra uno speedup di 1,42 da 2 a 4 worker, con efficienza 0,71. La separazione fra componente fissa e componente distribuita chiarisce che il limite non risiede nell'algoritmo ma nei costi fissi di inizializzazione e di accesso allo storage, che con tempi di esecuzione così contenuti rappresentano una quota rilevante del totale.

Codice sorgente: <https://github.com/saracasa/EarthquakeApplication.git>